Lista de exercícios - Métodos de Ordenação

Fácil

1. Organizando uma gaveta de documentos manualmente, um funcionário procedeu da seguinte forma: Considerou o primeiro documento como ordenado, e em seguida verificou em sequência, os próximos, colocando-os no início em posição ordenada, em relação aos elementos já vistos. Com qual tipo de ordenação estudado esse procedimento se parece?

2. Cite as principais características dos métodos ShellSort e HeapSort.

3. Os valores de Gap do método ShellSort, podem ultrapassar o tamanho do vetor à ser ordenado? Explique o porquê.

4. "Dividir para conquistar", é uma característica de qual método de ordenação? Cite outras características desse método.

5. Uma das vantagens da ordenação de conjuntos, está na facilitação da busca de elementos usando a busca binária. Qual a é sua vantagem em relação a busca sequencial? Explique por que não é possível ser usada em um conjunto não ordenado.

6. Em quantos passos é possível achar o elemento 11 no conjunto [1 3 4 5 7 9 11 13 85], usando a busca sequencial? E a binária?

7. Com qual método o ShellSort se assemelha quando o valor de Gap é igual à 1?

8. Faça o teste de mesa para o vetor [71 9 2 8 7 3 21], usando os métodos de ordenação que você conhece.

Médio

9. Projete o algoritmo mais eficiente que você conseguir para encontrar a mediana de um vetor de n elementos. Análise sua complexidade.

10. Crie um gráfico de 5 pontos, que mostre a quantidade de comparações feitas/números de elementos usados, na ordenação por inserção no pior caso. Use a implementação em c para auxiliar. Ex: [4 3 2 1] => [1 2 3 4] usou 6 comparações (passou 6 vezes pelo laço interno).

11. Modifique o código do método de ordenação por bolha para que ele transforme-se no método de ordenação por pedra, ou seja, o elemento desce

em vez de subir no vetor ordenado

12. Uma *ordenação por contagem* de um vetor *x* de tamanho *n* é executada da seguinte forma: declare um vetor *count* e defina *count*[*i*] como o número de elementos menores que *x*[*i*]. Em seguida, coloque *x*[*i*] na posição *count*[*i*] de um vetor de saída (leve em consideração a possibilidade de elementos iguais). Escreva uma função para ordenar um vetor x de tamanho n usando esse método.

13. Perdido em uma terra muito distante, você se encontra em frente a um muro de comprimento infinito para os dois lados (esquerda e direita). Em meio a uma escuridão total, você carrega um lampião que lhe possibilita ver apenas a porção do muro que se encontra exatamente à sua frente (o campo de visão que o lampião lhe proporciona equivale exatamente ao tamanho de um passo seu). Existe uma porta no muro que você deseja atravessar. Supondo que a mesma esteja a **n** passos de sua posição inicial (não se sabe se à direita ou à esquerda), elabore um algoritmo para caminhar ao longo do muro que encontre a porta o mais rápido possível pra qualquer **n**. Considere que cada passo demora o mesmo tempo.

Difícil/Grande

14. Dado um conjunto de *n* inteiros distintos e um inteiro positivo $k \leq n$: a. Proponha um algoritmo que imprime os k menos elementos do conjunto (em qualquer ordem) em tempo O(n). b. Suponha agora que queremos imprimir os k menos em ordem crescente. É ainda possível fazer isso em tempo O(n) para quaisquer valores $k \leq$ n?

15. A *ordenação por transposição de par-impar* ocorre da seguinte maneira. Percorra o vetor várias vezes. Na primeira passagem compare *x*[*i*] com *x*[*i*+1] para todo i ímpar. Na segunda passagem compare *x*[*i*] com *x*[*i*+1] para todo i par. Toda vez que *x*[*i*] > *x*[*i*+1] troque os dois. Continue alternando dessa maneira até que o vetor esteja ordenado. Qual a condição para o término da ordenação? Escreva uma função para implementar essa ordenação.

16. Implemente um programa que simule o jogo do Bingo. O programa deve gerar sucessivamente números aleatórios compreendidos entre 1 e 90 até que o usuário digite o caractere $. Para facilitar a verificação da cartela vencedora, o programa deve apresentar a lista ordenada de todos os números já sorteados

após o sorteio de um novo número. Use ordenação por inserção.

17. Escreva um algoritmo que ordena de forma crescente as letras pela frequência de ocorrência em um texto informado pelo usuário através do teclado. Utilize um dos métodos de ordenação.